

BUG REPORT

Project 2 — API Testing with Postman

Restful Booker API | restful-booker.herokuapp.com

Tester: Mass Frat | Date: May 2026 | Tool: Postman | Total Bugs Found: 5

1. Project Overview

This bug report documents all defects identified during manual API testing of the Restful Booker API as part of Project 2 — API Testing with Postman. Testing was conducted by Mass Frat in May 2026 using Postman as the primary testing tool.

The Restful Booker API is a practice hotel booking REST API. It exposes seven endpoints covering CRUD operations for hotel bookings. A total of 15 test cases were designed and executed across three test types: happy path, negative, and edge case testing.

2. Test Execution Summary

Total Test Cases	Passed	Failed	Bugs Found	Pass Rate
15	10	5	5	67%

Results by Test Type:

Test Type	Test Cases	Passed	Failed
Happy Path	9 (TC001–TC009)	9	0
Negative	3 (TC010–TC012)	1	2
Edge Case	3 (TC013–TC015)	0	3

3. Failed Test Cases Reference

The following test cases failed during execution. TC011 is included for reference as it passed.

TC ID	Description	Expected Status	Actual Status	Result
TC010	Create booking with missing last name field	400 Bad Request	500 Internal Server Error	✗ FAIL
TC011	Update booking with invalid auth token	403 Forbidden	403 Forbidden	☑ PASS
TC012	Delete a non-existent booking ID	404 Not Found	405 Method Not Allowed	✗ FAIL
TC013	Create booking with total price of zero	400 Bad Request	200 OK	✗ FAIL
TC014	Create booking with identical check-in and check-out dates	400 Bad Request	200 OK	✗ FAIL
TC015	Create booking with 50+ character first name	400 Bad Request	200 OK	✗ FAIL

4. Severity Guide

Symbol	Level	Definition
	Critical	System is unusable. Data loss or security breach possible.
	High	Major feature is broken. Needs urgent fix before release.
	Medium	Feature is impacted but a workaround exists.
	Low	Minor issue. No impact on core functionality.

5. Detailed Bug Reports

BUG-001 — High Severity

BUG-001 — POST /booking Returns 500 Internal Server Error When Required Field is Missing	
Severity: High Priority: High Endpoint: POST /booking Method: POST Test Case: TC010	
Environment	Chrome 148 / Windows 11 / Postman / May 2026
Steps to Reproduce	1. Open Postman and create a new POST request to <code>https://restful-booker.herokuapp.com/booking</code> 2. Go to the Headers tab and add: <code>Content-Type: application/json</code> 3. Go to the Body tab, select raw and set format to JSON 4. Enter the JSON body below — deliberately omitting the "last name" field: <code>{"first name": "John", "total price": 150, "deposit paid": true, "booking dates": {"check-in": "2025-01-01", "check-out": "2025-01-07"}}</code> 5. Click Send and observe the response status code and body
Expected Result	The API should detect the missing required field "last name" and return 400 Bad Request with a descriptive error message. The booking should NOT be created.
Actual Result	✗ The API returned 500 Internal Server Error. The server crashed instead of gracefully rejecting the request. No meaningful error message was returned.

BUG-002 — Medium Severity

BUG-002 — DELETE /booking/{id} Returns 405 Instead of 404 for Non-Existent Booking ID	
Severity: Medium Priority: Medium Endpoint: DELETE /booking/{id} Method: DELETE Test Case: TC012	
Environment	Chrome 148 / Windows 11 / 0Postman / May 2026
Steps to Reproduce	1. Run POST /auth to obtain a valid token and save it to the <code>{{token}}</code> environment variable 2. Create a new DELETE request to <code>https://restful-booker.herokuapp.com/booking/999999</code> (999999 is a non-existent ID) 3. Go to the Headers tab and add: <code>Cookie: token={{token}}</code> 4. Leave the request body empty 5. Click Send and observe the response status code
Expected Result	The API should return 404 Not Found to indicate no booking exists with that ID. This is the standard HTTP convention for a missing resource.
Actual Result	✗ The API returned 405 Method Not Allowed. This incorrectly implies the DELETE method itself is not permitted — which is misleading since DELETE is a valid documented method on this endpoint.

BUG-003 — Medium Severity

BUG-003 — POST /booking Accepts a Total Price of Zero With No Business Logic Validation	
Severity: Medium Priority: Medium Endpoint: POST /booking Method: POST Test Case: TC013	
Environment	Chrome 148 / Windows 11 / Postman / May 2026
Steps to Reproduce	1. Open Postman and create a new POST request to https://restful-booker.herokuapp.com/booking 2. Go to the Headers tab and add: Content-Type: application/json 3. Go to the body tab, select raw and set format to JSON 4. {"first name": "Zero", "last name": "Price", "total price": 0, "deposit paid": false, "booking dates": {"check-in": "2025-03-01", "checkout": "2025-03-05"}, "additional needs": "None"} 5. Click Send and observe the response
Expected Result	The API should return 400 Bad Request. A hotel booking with a total price of zero is not a valid business transaction.
Actual Result	✗ The API returned 200 OK and created the booking with a total price of zero, assigning it a new booking id. No validation error was returned.

BUG-004 — Medium Severity

BUG-004 — POST /booking Allows Booking With Identical Check-in and Check-out Dates	
Severity: Medium Priority: Medium Endpoint: POST /booking Method: POST Test Case: TC014	
Environment	Chrome 148 / Windows 11 / Postman / May 2026
Steps to Reproduce	1. Open Postman and create a new POST request to https://restful-booker.herokuapp.com/booking 2. Go to the Headers tab and add: Content-Type: application/json 3. Go to the Body tab, select raw and set format to JSON 4. {"first name": "Same", "last name": "Day", "total price": 100, "deposit paid": true, "booking dates": {"check-in": "2025-05-01", "check-out": "2025-05-01"}, "additional needs": "None"} 5. Click Send and observe the response
Expected Result	The API should return 400 Bad Request. A booking where check-out is the same as check-in represents a zero-night stay and is not a valid booking.
Actual Result	✗ The API returned 200 OK and created the booking with identical check-in and check-out dates, assigning it a new booking id. No validation error was returned.

BUG-005 — Medium Severity

[illegible]

6. Retest Tracker

Once the development team has applied fixes, all bugs must be retested to confirm the fixes work correctly and have not introduced any new issues.

Bug ID	Title	Status	Fix Version	Retest Date		Retested By	Result
BUG-001	POST /booking 500 on missing required field	Open	—	—		Mass Frat	—
BUG-002	DELETE returns 405 on non-existent booking ID	Open	—	—		Mass Frat	—
BUG-003	POST /booking accepts zero total price	Open	—	—		Mass Frat	—
BUG-004	POST /booking allows identical check-in and check-out dates	Open	—	—		Mass Frat	—
BUG-005	POST /booking accepts 50+ character first name	Open	—	—		Mass Frat	—

Retest Process:

1. Receive notification from the development team that a bug fix has been deployed
2. Re-run the original failing test case in Postman using the exact same steps and request body
3. Verify the actual result now matches the expected result and the correct status code is returned
4. Run surrounding test cases to check for regression — confirm no previously passing tests have broken
5. Update the retest status to Closed if confirmed fixed, or Reopened if the bug still exists

